The difference co-array in the near field: why it is weaker here, and how it could be made to work

Page 1 — Why the difference co-array is not effective in the near field

What the difference co-array assumes. The whole trick of the difference co-array rests on one fact: for a signal arriving as a *flat* (plane) wave, the phase an antenna sees is a *straight-line* function of its position,

$$\phi_m \;=\; \frac{2\pi}{\lambda}\, x_m \sin\theta,$$

where x_m is the position of antenna m, λ the wavelength, and θ the arrival angle. When you form the covariance of the received signal, every entry depends only on the *difference* $x_m - x_n$:

$$[\mathbf{R}]_{m,n} \;\propto\; e^{\,j\frac{2\pi}{\lambda}(x_m - x_n)\sin\theta}.$$

Because only the difference matters, two physical antennas at x_m and x_n behave exactly like one *virtual* antenna sitting at $x_m - x_n$. That is how a handful of real antennas generate a much longer "virtual" array. **This only works because the phase is linear in position.**

The near-field problem in one line. In the near field the wavefront is *curved*, not flat, so the phase is *no longer a straight line* in x_m — it has an extra *squared* term that depends on the user's *distance* (range), not just the angle:

$$\phi_m \;\approx\; \frac{2\pi}{\lambda}\Big(\underbrace{x_m \sin\theta}_{\text{linear: angle}} \;-\; \underbrace{\frac{x_m^2\cos^2\theta}{2r}}_{\text{quadratic: range}}\Big).$$

Why that breaks the virtual array. Put that phase into the covariance. The entry $[\mathbf{R}]_{m,n}$ now depends on $x_m^2 - x_n^2$ as well as $x_m - x_n$. The term $x_m^2 - x_n^2 = (x_m - x_n)(x_m + x_n)$ mixes in the *sum* $x_m + x_n$, so the covariance *no longer* depends on the difference alone. The clean "virtual antenna at $x_m - x_n$" picture collapses:

- Two different physical pairs that share the same *difference* $x_m - x_n$ no longer produce the *same* virtual-antenna phase, because their *sums* differ. The virtual array is no longer a uniform line array, so MUSIC and the other subspace methods built on it lose resolution or point to the wrong angle.
- Near-field antennas also receive *different signal strengths* (an antenna closer to the user gets a stronger signal). The plane-wave model assumes equal strength across the array; that assumption fails too, which further corrupts the covariance the co-array is built from.

What this means for us. In our deployment the users that matter for the sparse array are the *crowded, close-in* users — exactly the ones on near-field links. For those users the difference co-array is operating outside the regime it was designed for, so it gives at best a rough angle and cannot, by itself, deliver the range. This is precisely what the simulation showed: the co-array helped a little (it is not useless), but it did not beat our ECA + CAPE, and on near-field links it behaves closer to a plain array than to its promised N^2-sized virtual array.

What "set of sub-arrays" means in Pinto & de Lamare's paper, and why one array is not enough. After building the virtual array, Pinto and de Lamare do *not* run MUSIC on it directly. The reason is mathematical: the vectorized covariance $\mathbf{z} = \mathrm{vec}(\mathbf{R})$ behaves like a *single snapshot* of the virtual array, and a single snapshot makes all the users look perfectly *coherent*. A subspace method then sees a *rank-1* covariance and *cannot separate* more than one user — one virtual array alone fails.
The fix is *spatial smoothing*: slide a shorter window along the long virtual array to cut it into many *overlapping sub-arrays*, and *average* their individual covariances. Averaging over this *set of sub-arrays* rebuilds the rank back up to the number of users, so the signal subspace can be recovered. So the "set of sub-arrays" is not a hardware choice and not several physical panels — it is the collection of overlapping windows *inside the one virtual array*, and it is *required*: with a single (full) sub-array the method mathematically collapses to rank 1 and resolves nothing.

Page 2 — How the difference co-array could be made to work in the near field

The standard difference co-array is weak in the near field because it only uses the *difference* lags and assumes a flat wavefront. The three ideas below change *how* the co-array is formed or used so that it respects the near-field physics. (These are implemented in the updated code as Ideas 2, 3 and 4 and compared against the original CAPE.)

Idea 2 — Cross-AP (distributed) co-array. Instead of building the virtual array *inside one access point (AP)*, build it *across APs*, from the combined set of all AP antenna positions. Because our CAPE technique already fuses several APs looking at the same user from different sides, the combined "virtual aperture" spans the whole room, not one small panel. *How it dodges the near-field problem:* a single large panel is what makes the wavefront look curved; many *small* panels spread across the room each see an almost-flat wavefront, yet together they localise the user by where the angles cross. The APs are *not* phase-locked to each other, so the per-AP spectra are combined in the *power* (non-coherent) domain, not coherently. *Benefit:* resolution set by the *network* size, not the panel size.

Idea 3 — Joint sum-and-difference co-array. The near-field phase has *both* a linear (angle) part and a squared (range) part, and the squared part naturally produces *sum* lags $x_m + x_n$ — the very lags the ordinary difference co-array throws away. Idea 3 *keeps both*: it builds the difference co-array from the usual covariance *and* a sum co-array from the complementary moment, then lets *both* sets of angles propose user positions. *How it dodges the near-field problem:* instead of fighting the sum term, it *uses* it — the sum lags are the ones that carry the range information, so difference + sum gives angle *and* range from the same data. *Caveat built into the code:* the sum co-array only carries information when the data is not perfectly circular; with the deterministic training pilots we use it does carry structure, but with purely random QPSK symbols the sum part averages to zero — so Idea 3 helps most during pilot-based estimation.

Idea 4 — Hybrid-field reconfigurable co-array. Our network is *mixed*: some AP–user links are near-field (close users) and some are far-field (distant users). The difference co-array is *exactly valid* on the far-field links and *invalid* on the near-field ones — so do not apply the same processing to both. Idea 4 *switches per link*: on a far-field link it uses the full difference co-array with a plane-wave (angle-only) model; on a near-field link it uses the symmetric part of the array with a *range-aware* search. The switch uses the Rayleigh-distance test we already compute for every link. *How it dodges the near-field problem:* each link is processed with the model that is actually correct for it, so the co-array is never used where it is invalid.

Summary. The one-line reason the difference co-array underperforms here is that it assumes a flat wavefront, while our close-in users produce curved wavefronts whose phase carries an extra range-dependent (squared) term. The three ideas address this in three different ways: Idea 2 makes every panel small enough that the wavefront looks flat again; Idea 3 keeps the sum lags that the curvature produces instead of discarding them; Idea 4 applies the co-array only on the links where it is valid. The updated code implements all three and plots them against the original ECA + CAPE so the gain (or loss) of each can be read directly.